

GQA & MQA: Sharing the K/V Heads, Worked Out by Hand

CUTTING THE KV CACHE 8–64× BY SHARING KEY/VALUE HEADS — THE MEMORY MATH, BY HAND

What this document does. It derives the KV-cache size formula $KV_{\text{bytes}} = 2LH_{kv}d_{\text{head}}T \cdot (\text{dtype})$ and uses it to compare the three head-sharing variants: Multi-Head Attention (MHA, $H_{kv} = H$), Grouped-Query Attention (GQA- g , $H_{kv} = H/g$), and Multi-Query Attention (MQA, $H_{kv} = 1$). The reduction ratio is exactly H_{kv}/H . The worked example sizes Llama-3-70B’s actual GQA-8 at $T = 6200$ tokens and gets 2.03 GB per request (✓), versus 16.3 GB for MHA and 0.25 GB for MQA. We close with the accuracy tradeoff that explains why Llama-3 chose GQA-8 and not MQA. Every figure comes from Appendix A (`m03_t03_gqa_mqa_engine.py`).

The one idea. FlashAttention (Topic 2) made attention *compute* cheap. But during *decode*, the binding constraint is reading the entire KV cache from HBM at every single step. With full MHA that cache is enormous and grows with every token. GQA and MQA shrink it by giving many query heads a *shared* set of key/value heads — buying an order of magnitude more concurrent requests per GPU for almost no accuracy loss.

CONCEPTS COVERED, IN ORDER

why decode’s enemy is the KV cache; the KV-bytes formula and each of its factors; the three variants and their H_{kv}/H reduction ratios; a Llama-3-70B worked example at $T = 6200$; concurrency on one H100; and the accuracy tradeoff (why GQA-8, not MQA).

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	Why decode’s real enemy is the KV cache	2
2	Step 1 — The KV-cache size formula	2
3	Step 2 — The three variants	2
4	Step 3 — Worked example: Llama-3-70B at $T = 6200$	3
5	Step 4 — What it buys: concurrency per GPU	3
6	Step 5 — The accuracy tradeoff: why GQA-8, not MQA	4
7	The argument, at a glance	4
7.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 Why decode’s real enemy is the KV cache

In decode the model generates one token at a time, and at every step it must attend the new query against *all* past keys and values. To avoid recomputing them, K and V for every past token are stored in the **KV cache** and re-read from HBM each step.

Intuition

The query for the new token is a single $[1 \times d]$ vector — trivial. But K and V are $[T \times d]$: the entire history. Decode reads T tokens of cache to produce 1 token of output, so it is overwhelmingly memory-bound, and the cache size — not the FLOPs — decides how many requests you can fit on a GPU. Shrinking the KV cache is the single highest-leverage move for decode throughput, and that is exactly what GQA and MQA do.

2 Step 1 — The KV-cache size formula

The cache stores, for every layer, every KV head, and every token, a key vector and a value vector of dimension d_{head} .

KV bytes per request

$$\text{KV}_{\text{bytes}} = 2 \times L \times H_{kv} \times d_{\text{head}} \times T \times (\text{dtype bytes}).$$

Factor	Meaning
2	one each for K and V
L	number of transformer layers
H_{kv}	number of KV heads ($= H$ for MHA; H/g for GQA; 1 for MQA)
d_{head}	head dimension (here 128)
T	context length in tokens
dtype	bytes per element: 2 (FP16/BF16), 1 (FP8), 0.5 (INT4)

Only one factor changes between the three variants: H_{kv} . Everything else — the number of *query* heads H , d_{head} , L , T — is identical. So the reduction ratio is simply H_{kv}/H .

3 Step 2 — The three variants

Variant	H_{kv}	ratio = H_{kv}/H	idea
MHA	$H = 64$	1	every query head has its own K, V head
GQA- g	H/g	$1/g$	query heads share K/V in groups of g
MQA	1	$1/H$	all query heads share a single K, V head

For Llama-3-70B ($H = 64$), “GQA-8” means 8 KV heads ($H_{kv} = 8$), i.e. each KV head is shared by $64/8 = 8$ query heads — an $8\times$ reduction. MQA ($H_{kv} = 1$) is the extreme: a single shared KV head, a $64\times$ reduction.

Per-token cache for each variant (all 80 layers, FP16)

Using per-token = $2 L H_{kv} d_{\text{head}} \cdot 2$:

$$\text{MHA } (H_{kv}=64) : 2 \cdot 80 \cdot 64 \cdot 128 \cdot 2 = 2,621,440 \text{ B} = 2.621 \text{ MB/token}$$

$$\text{GQA-8 } (H_{kv}=8) : 2 \cdot 80 \cdot 8 \cdot 128 \cdot 2 = 327,680 \text{ B} = 0.328 \text{ MB/token}$$

$$\text{GQA-4 } (H_{kv}=4) : 2 \cdot 80 \cdot 4 \cdot 128 \cdot 2 = 163,840 \text{ B} = 0.164 \text{ MB/token}$$

$$\text{MQA } (H_{kv}=1) : 2 \cdot 80 \cdot 1 \cdot 128 \cdot 2 = 40,960 \text{ B} = 0.041 \text{ MB/token}$$

4 Step 3 — Worked example: Llama-3-70B at $T = 6200$

Multiply the per-token cost by the context length $T = 6200$.

Full per-request KV cache, $T = 6200$, FP16

$$\text{MHA} : 2 \cdot 80 \cdot 64 \cdot 128 \cdot 6200 \cdot 2 = 16,252,928,000 \text{ B} = 16.25 \text{ GB}$$

$$\text{GQA-8} : 2 \cdot 80 \cdot 8 \cdot 128 \cdot 6200 \cdot 2 = 2,031,616,000 \text{ B} = \mathbf{2.03 \text{ GB}} \checkmark$$

$$\text{GQA-4} : 2 \cdot 80 \cdot 4 \cdot 128 \cdot 6200 \cdot 2 = 1,015,808,000 \text{ B} = 1.02 \text{ GB}$$

$$\text{MQA} : 2 \cdot 80 \cdot 1 \cdot 128 \cdot 6200 \cdot 2 = 253,952,000 \text{ B} = 0.25 \text{ GB}$$

Variant	H_{kv}	KV/request	ratio vs MHA	vs MHA
MHA	64	16.25 GB	1.000	1×
GQA-8	8	2.03 GB	0.125	8× less
GQA-4	4	1.02 GB	0.0625	16× less
MQA	1	0.25 GB	0.0156	64× less

The headline

A single $T = 6200$ Nexus request needs 16.3 GB of KV cache under MHA — a fifth of an entire H100 for *one* user's history. Llama-3-70B's actual GQA-8 cuts that to 2.03 GB, an 8× reduction, with the reduction ratio being exactly $H_{kv}/H = 8/64 = 1/8$. MQA would cut it 64× to 0.25 GB.

5 Step 4 — What it buys: concurrency per GPU

KV-cache size translates directly into how many simultaneous requests fit. On one H100 (80 GB HBM) with ~ 37 GB of W4-quantized weights, ≈ 43 GB remains for KV:

Concurrent $T = 6200$ requests in 43 GB

$$\text{concurrency} = \frac{43 \text{ GB}}{\text{KV per request}}.$$

$$\text{MHA} : 43/16.25 \approx 2.6 \text{ requests}$$

$$\text{GQA-8} : 43/2.03 \approx \mathbf{21} \text{ requests}$$

$$\text{GQA-4} : 43/1.02 \approx 42 \text{ requests}$$

$$\text{MQA} : 43/0.25 \approx 169 \text{ requests}$$

Intuition

GQA-8 turns “can barely fit 2 users” into “comfortably serve 20.” That single architectural choice is an order of magnitude of request concurrency — and therefore of throughput-per-dollar — at essentially no quality cost. This is why GQA, not MHA, is the default in every open model family from Llama-3 forward.

6 Step 5 — The accuracy tradeoff: why GQA-8, not MQA

MQA gives $64\times$ reduction — strictly more than GQA-8’s $8\times$. So why didn’t Llama-3 use it? Because sharing is not free in quality.

Variant	Δ MMLU (70B-class)	character
MHA	0.0	baseline
GQA-8	−0.2	negligible — Llama-3 default
GQA-4	−0.4	small
MQA	−1.2	noticeable degradation

The expressiveness argument

Different query heads learn to attend to genuinely different aspects of the past — one tracks syntax, another long-range coreference, another local n-grams. Each head’s “question” is most useful against keys/values *specialized* to that question. MQA forces all 64 heads to read from one shared K/V projection, collapsing that diversity and measurably hurting quality (~ 1.2 MMLU on a 70B model). GQA-8 keeps 8 distinct K/V projections — enough specialization that quality is essentially preserved (−0.2) — while still cutting memory $8\times$. It is the sweet spot of the memory/quality curve.

The design rule

Reduction ratio is H_{kv}/H , so smaller H_{kv} is always less memory — but each halving of H_{kv} costs expressiveness. GQA-8 sits where the memory curve is already $8\times$ down but the quality curve has barely moved. Drop to GQA-16 only if quality is paramount; drop to MQA only if memory is so tight that −1 MMLU is acceptable; otherwise, if you need more than $8\times$, switch architectures to MLA (Topic 4) rather than pushing H_{kv} to 1.

7 The argument, at a glance

#	Step	Result
1	Decode enemy	every step re-reads the whole KV cache from HBM
2	Formula	$KV = 2LH_{kv}d_{\text{head}}T(\text{dtype})$
3	Variants	ratio = H_{kv}/H : MHA 1, GQA- g $1/g$, MQA $1/H$
4	Example	GQA-8 @ $T=6200 = 2.03$ GB vs MHA 16.25 GB ✓
5	Concurrency	GQA-8 fits ~ 21 reqs vs MHA’s ~ 2.6 on one H100
6	Accuracy	GQA-8 −0.2 MMLU vs MQA −1.2 $\Rightarrow$ pick GQA-8

7.1 Equation cheat-sheet

KV bytes	$KV = 2 L H_{kv} d_{\text{head}} T (\text{dtype})$
Reduction ratio	H_{kv}/H (MHA 1, GQA- g $1/g$, MQA $1/H$)
Per-token	$2 L H_{kv} d_{\text{head}} (\text{dtype})$
Concurrency	$(\text{HBM} - \text{weights})/\text{KV per request}$

Appendix A — Reference implementation

Every number above is produced by `m03_t03_gqa_mqa_engine.py`. Change H_{kv} , T , or the dtype to size any model/serving configuration.

```

1  """
2  Reference implementation for: Module 03 - Topic 3
3  "GQA & MQA: Sharing the K/V Heads, Worked Out by Hand."
4
5  KV-cache sizing for MHA vs GQA-g vs MQA, with the Llama-3-70B worked example.
6
7  Convention: 1 GB = 1e9 bytes, 1 MB = 1e6 bytes, 1 KB = 1e3 bytes (decimal).
8  Every number printed here is transcribed (rounded) into the worked-by-hand PDF.
9  """
10
11 # -----
12 # 0. Model + serving constants (Llama-3-70B class, the Nexus target).
13 # -----
14 L      = 80          # transformer layers
15 H      = 64          # query heads (and MHA K/V heads)
16 d_head = 128         # head dimension
17 dtype_b = 2         # FP16 KV, bytes per element
18 T      = 6200        # context length (Nexus prompt)
19
20 print("== 0. Constants ==")
21 print(f"L={L} H={H} query heads d_head={d_head} dtype={dtype_b}B (FP16) T={T} tokens")
22
23 # -----
24 # 1. The KV-cache formula.
25 #   KV_bytes = 2 * L * H_kv * d_head * T * dtype_bytes
26 #   (the 2 is one each for K and V)
27 # -----
28 def kv_bytes(H_kv, T_):
29     return 2 * L * H_kv * d_head * T_ * dtype_b
30
31 def kv_per_token(H_kv):
32     return 2 * L * H_kv * d_head * dtype_b
33
34 print("\n== 1. KV formula: KV_bytes = 2 * L * H_kv * d_head * T * dtype_bytes ==")
35
36 # -----
37 # 2. Per-token cache for each variant.
38 # -----
39 variants = {
40     "MHA (H_kv=64)" : H,          # H_kv = H
41     "GQA-8 (H_kv=8)" : H // 8,    # 8 KV heads (Llama-3-70B actual)
42     "GQA-4 (H_kv=4)" : H // 16,   # 4 KV heads
43     "MQA (H_kv=1)"  : 1,          # 1 KV head
44 }
45 print("\n== 2. KV per token (all 80 layers) ==")
46 for name, hkv in variants.items():
47     pt = kv_per_token(hkv)

```

```

48     print(f"   {name:16s}: 2*{L}*{hkv}*{d_head}*{dtype_b} = {pt:>10,d} B = {pt/1e6:.3f} MB/token")
49
50 # -----
51 # 3. Full cache for one T=6200 request, and reduction ratios vs MHA.
52 # -----
53 print(f"\n== 3. Full KV cache for one T={T} request, ratio vs MHA ==")
54 mha = kv_bytes(H, T)
55 for name, hkvt in variants.items():
56     b = kv_bytes(hkvt, T)
57     print(f"   {name:16s}: {b:>14,d} B = {b/1e9:.3f} GB    ratio vs MHA = H_kv/H = {hkvt/H:.4f}  "
58           f"({H/hkvt:.0f}x less)")
59
60 # -----
61 # 4. The headline numbers used in the document.
62 # -----
63 print("\n== 4. Headline ==")
64 gqa8 = kv_bytes(H // 8, T)
65 mqa = kv_bytes(1, T)
66 print(f"MHA   : {mha/1e9:.3f} GB per request")
67 print(f"GQA-8: {gqa8/1e9:.3f} GB per request    ({mha/gqa8:.0f}x less than MHA)")
68 print(f"MQA   : {mqa/1e9:.3f} GB per request    ({mha/mqa:.0f}x less than MHA)")
69
70 # -----
71 # 5. Concurrency on one H100 (80 GB) with AWQ-W4 weights (~37 GB).
72 # -----
73 print("\n== 5. Concurrency on one H100 (80 GB), W4 weights ~37 GB ==")
74 hbm      = 80e9
75 weights  = 37e9
76 free_kv  = hbm - weights
77 for name, hkvt in variants.items():
78     per_req = kv_bytes(hkvt, T)
79     print(f"   {name:16s}: {free_kv/per_req:5.1f} concurrent T={T} requests fit in {free_kv/1e9:.0f} GB")
80
81 # -----
82 # 6. Accuracy tradeoff (published ablations, 70B-class; MMLU points lost).
83 # -----
84 print("\n== 6. Accuracy tradeoff (MMLU points vs MHA baseline) ==")
85 acc = [("MHA", 0.0, "baseline"),
86        ("GQA-8", -0.2, "negligible -- Llama-3 default"),
87        ("GQA-4", -0.4, "small"),
88        ("MQA", -1.2, "noticeable; heads forced to share one K/V")]
89 for name, dmmlu, note in acc:
90     print(f"   {name:6s}: {dmmlu:+.1f} MMLU    {note}")

```

— END OF DOCUMENT —